Lecture 6 · Machine-Level Programming III: Procedures

10/15 (목) · 녹화 강의 · 슬라이드 "Lecture 6 Procedures" (PDF 38~39, 49~72) + 수업 녹취. PDF 2~37, 40~48, 73~74는 녹화가 없어 슬라이드만 보고 정리

1 Procedure 호출에 필요한 것

세 가지 메커니즘과 ABI

프로시저(함수)를 부르고 돌아오는 데는 세 가지가 필요:

- **Passing control**: 부른 프로시저 code의 시작으로 가고, 끝나면 호출한 자리로 돌아옴
- **Passing data**: 인자와 반환값
- **Memory management**: 프로시저가 실행되는 동안 공간을 할당하고, return하면 해제

함수 호출은 ISA가 정의한 call, ret, push, pop 등의 기계어 명령어를 이용해 구현된다. **ABI (Application Binary Interface)**는 함수 호출 규약으로서 인자와 반환값의 전달 위치, caller-saved와 callee-saved 레지스터, 스택 정렬 등의 규칙을 정한다. 구체적인 명령어 순서와 스택 프레임 구성은 컴파일러와 최적화에 따라 달라질 수 있다. x86-64는 필요한 메커니즘만 씀 (인자도 local 변수도 없는 프로시저는 그 부분을 구현하지 않아도 됨).
슬라이드 예제: P가 y = Q(x)로 Q를 부르고 print(y)로 이어감. Q 안의 local 변수 t와 배열 v[10]은 Q가 실행되는 동안만 필요한 공간.

- 강의 순서: Stack Structure → Calling Conventions (Passing Control, Passing Data, Managing Local Data) → Recursion & Pointers

2 x86-64 stack과 push, pop

stack과 %rsp

- stack은 stack discipline(LIFO, 나중에 들어간 것이 먼저 나옴)으로 관리하는 memory 영역. memory는 byte 배열, 영역마다 쓰임이 다름 (ABI 같은 정책 결정)
- **stack은 낮은 주소 쪽으로 자람**. %rsp는 stack에서 가장 낮은 주소, 즉 top 원소의 주소를 담음

pushq와 popq

- `pushq src`: src의 값을 읽음 → %rsp를 8 줄임 → %rsp가 가리키는 주소에 그 값을 씀
- `popq dst`: %rsp가 가리키는 주소에서 값을 읽음 → %rsp를 8 늘림 → 읽은 값을 dst에 저장
- **No cleanup**: pop 뒤에도 값은 memory에 남아 있고 %rsp가 어디까지가 유효한 stack인지를 정함
- pushq와 popq는 각각 하나의 기계어 명령어로 실행된다. 여기서 단일 명령어라는 뜻은 동기화 관점에서 원자적 메모리 연산이 보장된다는 뜻은 아니다.
- 8바이트씩 움직이는 것은 q(quad word)가 8바이트이기 때문
- push는 %rsp를 먼저 줄이고 쓰고, pop은 먼저 읽고 %rsp를 늘림 → 둘은 서로를 정확히 되돌림

3 call과 ret: 제어 이동

예제: multstore와 mult2

```
void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
long mult2(long a, long b) { long s = a * b; return s; }
```

```
0000000000400540 <multstore>:
  400540: push %rbx              # Save %rbx
  400541: mov  %rdx,%rbx         # Save dest
  400544: callq 400550 <mult2>  # mult2(x,y)
  400549: mov  %rax,(%rbx)       # Save at dest
  40054c: pop  %rbx              # Restore %rbx
  40054d: retq
0000000000400550 <mult2>:
  400550: mov  %rdi,%rax         # a
  400553: imul %rsi,%rax         # a * b
  400557: retq
```

- callq, retq는 call, ret의 disassembler 표기 (q는 64bit)
- multstore: push %rbx로 저장, mov %rdx,%rbx로 dest 보관, call 뒤 mov %rax,(%rbx)로 결과 저장, pop %rbx로 되돌림
- (보충) %rdx는 caller-saved라 call을 넘어 값이 남는다는 보장이 없어서 callee-saved인 %rbx에 dest를 둠

call과 ret이 하는 일

call label: return address를 stack에 push하고 label로 점프. return address = call 바로 다음 instruction의 주소.
ret: stack에서 return address를 pop하고 그 주소로 점프.

손계산 1 · call과 ret에서 %rsp와 %rip (시작: %rsp = 0x120, %rip = 0x400544 = callq 직전)

callq 400550 <mult2>: return address = $0x400549$ (다음 줄 주소, callq는 5바이트). %rsp = $0x120 - 8 = 0x118$, [0x118] = 0x400549, %rip = 0x400550
mov %rdi,%rax (0x400550) → %rip = 0x400553 · **imul %rsi,%rax** (0x400553) → %rip = 0x400557, %rsp는 계속 0x118
retq (0x400557): %rip = [0x118] = 0x400549, %rsp = $0x118 + 8 = 0x120$. call 직전과 %rsp가 같음 (call이 8 줄이고 ret이 8 늘림)
이어서 multstore: mov %rax,(%rbx) (0x400549) → pop %rbx (0x40054c): %rsp = 0x128 → retq (0x40054d): %rsp = 0x130, multstore를 부른 곳으로 복귀

① multstore 진입 직후 %rsp = 0x128 ([0x128]에 return address)

② push %rbx로 0x120

③ pop %rbx로 0x128 복귀. pop된 0x400549 값은 0x118 칸에 흐리게 남아 있음 (no cleanup)

4 인자와 반환값: Procedure Data Flow

System V AMD64에서 이 노트처럼 인자가 모두 정수·포인터형인 예제에서는 처음 여섯 인자를 %rdi, %rsi, %rdx, %rcx, %r8, %r9에 전달하고, 일곱 번째 이후 인자를 스택에 전달한다. 정수형·포인터 반환값은 %rax에 놓으며, 부동소수점·벡터 값은 별도의 XMM 레지스터 규칙을 따른다.

- multstore: x는 %rdi, y는 %rsi, dest는 %rdx · mult2: a는 %rdi, b는 %rsi. multstore가 받은 x, y가 같은 register에 그대로 있어서 옮기지 않고 바로 call
- mult2는 a×b를 %rax에 남기고, multstore는 그 값을 받아 mov %rax,(%rbx)로 dest에 저장
- dest(%rdx)만 call 전에 %rbx로 옮겨 둠. %rdx는 call을 넘어 보존된다는 보장이 없기 때문

손계산 2 · mult2의 값 흐름 (x = 6, y = 7 가정)

진입: %rdi = 6 (x), %rsi = 7 (y), %rdx = dest
push %rbx, mov %rdx,%rbx → %rbx = dest (호출자의 %rbx는 stack에 저장)
callq mult2: %rdi, %rsi 그대로 a = 6, b = 7 → mov → %rax = 6 → imul → %rax = $6 \times 7 = 42$ → retq
mov %rax,(%rbx): [dest] = 42 (*dest = t) · pop %rbx: 호출자의 %rbx 되돌림, retq

결과 ***dest = x × y = 42**

5 Stack 기반 언어와 Stack Frame

Recursion을 지원하는 언어

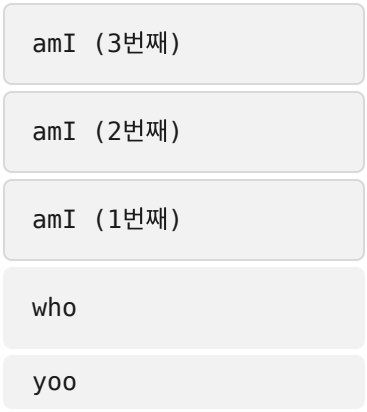
- C, Pascal, Java 같은 언어는 code는 **re-entrant**: 같은 프로시저의 실행이 동시에 여러 개 진행될 수 있고 앞선 호출이 끝나기 전에도 안전하게 실행됨
- → 각 실행의 상태(인자, local 변수, return pointer)를 둘 자리가 필요
- Stack discipline: 프로시저의 상태는 호출부터 return까지의 한정된 시간 동안만 필요하고, **callee가 caller보다 먼저 return**
- → stack은 frame 단위로 할당, frame 하나 = 프로시저 호출 하나의 상태

frame의 내용과 관리

- 내용: return 정보, local 저장 공간(필요하면), 임시 공간(필요하면)
- 관리: 진입 시 할당(set-up code, call이 push한 return address 포함), return 시 해제(finish code, ret이 pop하는 return address 포함)
- %rsp는 현재 frame의 맨 위, %rbp(frame pointer, optional)는 현재 frame의 아래쪽 경계, 그 아래(높은 주소)에 이전 frame들

call chain 예제: yoo, who, amI

가장 깊은 상태: yoo → who → amI → amI → amI



← %rsp (top, 가장 낮은 주소)

← 높은 주소 (bottom)

호출 순서

① yoo가 who() 호출
② who가 amI() 호출 (첫 번째)
③ amI가 자기 자신 호출 (재귀)
④ 한 번 더 재귀 → frame 5개

return은 역순: 가장 안쪽 amI부터
frame이 풀리고(who로 복귀), who의 두 번째
amI()가 해제된 자리에 새 frame을 쌓음

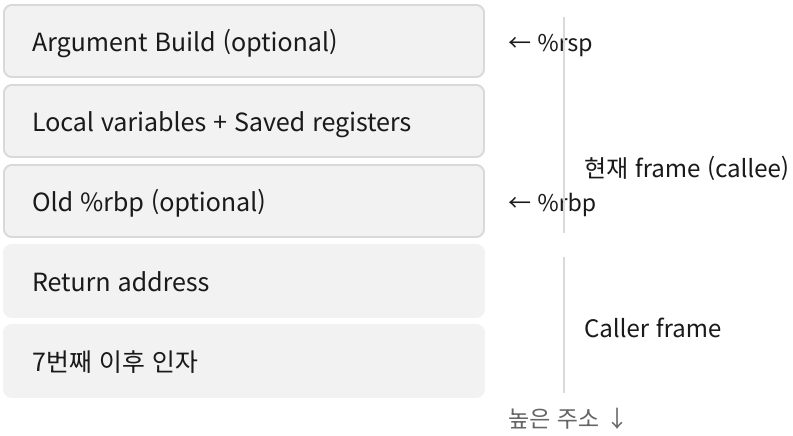
frame은 호출 깊이만큼 쌓였다가 return하면 풀리고, 해제된 자리는 다음 호출이 다시 씀 (슬라이드 25~36 그림 재구성)

손계산 3 · call chain의 stack frame과 %rsp (가정: yoo 실행 중 %rsp = 0x120, 호출 하나 = 16바이트)

slide	running	%rsp	frames
26	yoo	0x120	1
27	yoo who	0x110	2
28	yoo who amI	0x100	3
29	yoo who amI amI	0xf0	4
30	yoo who amI amI amI	0xe0	5
31~33	되감기 (amI×3 return)	0xf0 → 0x110	4 → 2
34	yoo who amI (두 번째)	0x100	3
35~36	who → yoo로 복귀	0x110 → 0x120	2 → 1

- 가장 깊을 때(슬라이드 30): $4 \times 16 = 64$ 바이트 (who 1개 + amI 3개), %rsp = 0x120 − 0x40 = 0xe0
- 호출 횟수: who 1번 + amI 4번 = 5번. call마다 frame이 하나 생기고 return마다 하나가 풀림
- 같은 호출 깊이에서는 %rsp가 같음: 슬라이드 28(첫 amI)과 34(두 번째 amI)가 모두 0x100 → 해제된 자리 재사용
- callee가 항상 caller보다 먼저 return하므로 frame을 LIFO로만 다뤄도 됨

x86-64/Linux stack frame 구조



%rsp는 맨 위(낮은 주소), return address와 7번째 이후 인자는 caller 쪽 (슬라이드 37/72 재구성)

- (보충) 7번째 이후 인자는 caller가 call 전에 stack에 두므로 return address보다 높은 주소에 있음 → callee 진입 직후 7번째 인자는 8(%rsp) (연습문제 9의 add10이 이 경우)

프로시저 호출과 스택 프레임 — Stack Frames

프로그램(Preset) `main → foo(8인자) → bar` 한 단계 뒤로 자동 재생 리셋

```
진행 0/20 · 활성 프레임 깊이 1
현재 활성 호출: main

한 단계씩 실행해 호출 규약과 프레임의 생명주기를 관찰합니다.
```

실행 순서 — Assembly Trace

```
00 main: 인자 1~6을 레지스터에 배정
01 subq $16,%rsp — outgoing args 공간
02 movq $7,0(%rsp); movq $8,8(%rsp)
03 call foo
04 foo: pushq %rbp
05 movq %rsp,%rbp
```

```
06 subq $16,%rsp
07 movq %rbx,0(%rsp) — callee-saved 저장
08 movq $99,%rbx — foo 본문
09 call bar
10 bar: pushq %rbp
11 bar: movq %rsp,%rbp
12 bar: movq $42,%rax
13 bar: leave
14 bar: ret
15 foo: movq 0(%rsp), %rbx
16 foo: movq $56,%rax
17 foo: leave
18 foo: ret
19 main: addq $16,%rsp
```

레지스터와 스택 정렬

레지스터	현재 값	역할
%rsp	0x0000000000008000	현재 스택 포인터(top)
%rbp	0x0000000000008000	현재 프레임 기준점
%rax	0x0000000000000000	반환값
%rbx	0x0000000000000000	callee-saved
%rdi	0x0000000000000000	인자 1
%rsi	0x0000000000000000	인자 2
%rdx	0x0000000000000000	인자 3
%rcx	0x0000000000000000	인자 4

레지스터	현재 값	역할
%r8	0x0000000000000000	인자 5
%r9	0x0000000000000000	인자 6

> %rsp mod 16 = 0

스택 지도 — 높은 주소에서 낮은 주소로 성장 ↓

▭ 활성 슬롯: 현재 호출에서 사용하는 값 ⬚ 해제됨: 함수 반환 후 값이 남아 있을 수 있음

현재 표시할 스택 슬롯이 없습니다.

System V AMD64 호출 규약
정수·포인터 인자 1~6: rdi, rsi, rdx, rcx, r8, r9. 7번째부터는 호출자가 스택에 배치합니다.
Caller-saved: rax, rcx, rdx, rsi, rdi, r8–r11. Callee-saved: rbx, rbp, r12–r15. 함수는 보존 의무가 있는 레지스터를 복구해야 합니다.
call 직전 %rsp는 16바이트 정렬, 일반적인 함수 진입 직후 %rsp mod 16 = 8입니다.
재귀 예제의 fact(n) 저장 슬롯은 각 호출의 n을 보존해 하위 호출 반환 후 곱셈에 사용합니다.
재귀 예제에서 %rdi는 call 직전에 준비된 인자(n-1)를 담고 있으며, 호출자는 반환 후 %rdi가 호출 전 값으로 유지된다고 가정할 수 없습니다(caller-saved).
회색 점선 슬롯은 반환된 프레임의 잔존 메모리를 나타내며, 활성 슬롯이 아닙니다. 실제 프로그램에서는 이후 호출이 해당 주소를 덮어쓸 수 있습니다.
교육용 고정 단계 추적이며 실제 명령어 인터프리터는 아닙니다.

6 Register를 건드려도 되는가

읽기와 쓰기: incr 예제

- register를 **읽는** 것은 문제없음 (값을 바꾸지 않음). **쓰는** 경우에는 프로그램이 깨지지 않는지 따져야 함
- caller와 callee가 제어와 데이터를 주고받는 관계라, 어느 쪽이 register를 바꾸든 상대의 실행이 깨지면 안 됨

```
incr:
    movq (%rdi), %rax   # x = *p
    addq %rax, %rsi     # y = x + val (%rsi가 val에서 y로 바뀜)
    movq %rsi, (%rdi)   # *p = y
    ret
```

- 수정되는 register: %rax(반환값 자리라 당연), %rsi(까다로운 쪽, call_incr가 넣어 둔 3000을 덮어씀). 이 경우는 괜찮음: call_incr가 call 뒤에 %rsi를 다시 읽지 않기 때문
- **문제는 callee가 덮어쓴 register를 caller가 return 뒤에 읽을 때** → 아래 yoo/who 예제

손계산 4 · call_incr의 주소와 %rsp (가정: 진입 직전 %rsp = 0x120, [0x120] = return address — 주소는 16바이트 정렬 규약을 생략한 가상 값, 실제 ABI 진입 주소 아님)

```
long call_incr() {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return v1 + v2;
}
```

subq $16, %rsp → %rsp = 0x110 · movq $15213, 8(%rsp) → [0x118] = 15213, v1의 주소는 $0x110 + 8 = 0x118$
movl $3000, %esi → %rsi = 3000 (32bit에 쓰면 상위 32bit는 0 · 일반적인 인코딩에서 movq $imm32보다 2바이트 짧음 — 5 vs 7바이트, 인코딩 선택에 따라 다를 수 있음) · leaq 8(%rsp), %rdi → %rdi = 0x118 (&v1)
call incr → %rsp = 0x108, [0x108] = 다음 줄 주소. incr 안: %rax = 15213, %rsi = $15213 + 3000 = 18213$, [0x118] = 18213. ret → %rsp = 0x110
addq 8(%rsp), %rax → %rax = $15213 + 18213 = 33426$ (return v1+v2: v1 = 새 값 18213, v2 = 옛 값 15213)
addq $16, %rsp → %rsp = 0x120 (두 칸 해제, 18213은 회색으로 남음) · ret → %rsp = 0x128

결과

반환값 = 33426. 가장 깊을 때(incr 진입 직후) %rsp = 0x108, 진입 직전보다 $16 + 8 = 24$ 바이트 아래

---

yoo / who: 덮어쓰기가 문제가 되는 경우

```
yoo:                    who:
    movq $15213, %rdx       subq $18213, %rdx
    call who                ...
    addq %rdx, %rax         ret
    ...
```

- yoo는 %rdx에 15213을 넣어 두고 who를 부른 뒤 그 값을 다시 읽음. who가 %rdx에서 18213을 빼 버리면 yoo의 15213은 사라짐 → 되돌릴 수 없음
- 함수마다 일일이 확인할 수는 없음 → **누가 보관할 책임을 지는지 약속이 필요**

손계산 5 · who가 %rdx를 덮어쓰면 (who는 %rax = 100 반환 가정)

기대한 값: $100 + 15213 = 15313$
실제: subq $18213, %rdx → %rdx = $15213 - 18213 = -3000$
addq %rdx, %rax → %rax = $100 + (-3000) = -2900$

결과 **15313이 아니라 −2900. who가 %rdx를 바꾼 만큼 결과가 틀어짐**

7 Register 저장 규약: caller-saved vs callee-saved

두 역할로 나눈 보관 책임

- **caller-saved**: caller가 call 전에 임시 값을 자기 stack frame에 저장하고 call 뒤에 되돌림 → callee는 이 register를 제약 없이 덮어써도 됨
- **callee-saved**: callee가 register를 쓰기 전에 값을 자기 stack frame에 저장하고 ret 전에 되돌림 → caller 입장에서는 call 전후로 값이 보존됨. **callee가 그 register를 수정할 때만** 저장·되돌림 책임이 생김

다른 방식도 있음: 아주 단순한 architecture는 call 전에 모든 register를 memory에 저장(전부 caller-save). x86은 두 방식을 섞어 register마다 책임을 나눔. ABI가 정하는 정책이라 architecture마다 다를 수 있음.

x86-64 Linux register 분류

분류	register	역할
caller-saved (callee가 마음대로 바꿔도 됨 → call 뒤 값이 남아 있다고 믿으면 안 됨)	%rax	반환값
	%rdi %rsi %rdx %rcx %r8 %r9	인자 6개
	%r10 %r11	특별한 용도가 없는 임시 register
callee-saved (call을 가로질러 값이 남음 → callee가 쓰려면 저장 후 되돌림)	%rbx %r12 %r13 %r14 %r15	
	%rbp	frame pointer로 쓸 수도 있음
	%rsp	특수한 형태: 함수에서 빠져나올 때 원래 값으로 복구

왜 이렇게 나뉘는가

- %rax: 반환값을 돌려주려면 callee가 반드시 덮어씀 → call 뒤에도 필요한 값이 있었다면 caller가 미리 저장
- 인자 register: callee에게 인자를 넘기려면 caller가 덮어써야 함 → 받은 인자를 다른 호출 뒤에도 써야 한다면 옛 값을 먼저 저장. 그래서 caller 쪽 책임
- 특별한 의미 없는 임시 register는 책임을 둘로 나눔: %r10, %r11은 caller 쪽, %rbx와 %r12~%r15는 callee 쪽
- **함수 하나가 caller이면서 callee**: 어떤 register는 caller로서 챙기고, 어떤 register는 callee로서 챙김
- %rsp와 %rbp: stack은 caller/callee 경계에 걸쳐 있음 → frame을 만든 callee가 정리하는 쪽이 자연스러움 (%rsp = 원래 값으로 복구, return address = caller의 context로 복귀)

callee-saved 예제: call_incr2

```
long call_incr2(long x) {
    long v1 = 15213;
    long v2 = incr(&v1, 3000);
    return x + v2;
}
```

- 문제: x는 %rdi로 들어오고 마지막 x + v2까지 보관돼야 하는데 incr에 첫 인자 &v1을 넘기려면 %rdi를 덮어써야 함. %rdi는 caller-saved라 보장 없음
- 해결: callee-saved register인 %rbx에 x를 둠 → incr가 %rbx를 쓰더라도 return 전에 되돌려 줌. 대신 내가 %rbx를 쓰려면 내 caller의 %rbx를 먼저 저장했다가 되돌림

```
call_incr2:
    pushq %rbx          # 내 caller의 %rbx를 stack에 저장
    subq  $16, %rsp     # local 변수 공간 확보
    movq  %rdi, %rbx    # x → %rbx (call을 넘어 보존)
    movq  $15213, 8(%rsp) # v1 = 15213
    movl  $3000, %esi   # 2번째 인자
    leaq  8(%rsp), %rdi # 1번째 인자 &v1
    call  incr
    addq  %rbx, %rax    # x + v2
    addq  $16, %rsp     # 확보한 공간 반환
    popq  %rbx          # 내 caller의 %rbx 되돌림
    ret
```

- 되돌리기는 저장의 역순(LIFO): addq $16, %rsp → popq %rbx → ret. 이때 %rsp가 return address를 가리켜야 ret이 제대로 돌아감
- 저장·되돌림 장소에 정해진 한 방법은 없음. 남는 register를 쓰는 편이 memory보다 빠름 (compiler 최적화)

손계산 6 · call_incr2의 stack과 register (가정: 진입 직전 %rsp = 0x120, [0x120] = R0, 내 caller의 %rbx = B0 — 주소는 16바이트 정렬 규약을 생략한 가상 값, 실제 ABI 진입 주소 아님)

pushq %rbx → %rsp = 0x118, [0x118] = B0 · subq $16, %rsp → %rsp = 0x108

movq %rdi, %rbx → %rbx = 100 · movq $15213, 8(%rsp) → [0x110] = 15213, v1의 주소 = $0x108 + 8 = 0x110$

movl $3000, %esi → %rsi = 3000 · leaq 8(%rsp), %rdi → %rdi = 0x110 (&v1)

call incr → %rsp = 0x100, [0x100] = 다음 줄 주소. incr 안: %rax = 15213, %rsi = $15213 + 3000 = 18213$, [0x110] = 18213. ret → %rsp = 0x108

addq %rbx, %rax → %rax = $15213 + 100 = 15313$

addq $16, %rsp → %rsp = 0x118 · popq %rbx → %rbx = B0 되돌림, %rsp = 0x120 · ret → %rsp = 0x128

결과

반환값 = x + (incr가 돌려준 원래 *p) = $100 + 15213 = 15313$. v1은 18213으로 바뀌지만 반환식 x + v2에는 쓰이지 않음

8 Recursion: pcount_r

```
long pcount_r(unsigned long x) {
    if (x == 0) return 0;
    else return (x & 1) + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl $0, %eax       # 반환값 = 0 (종료 경로 대비)
    testq %rdi, %rdi    # x & x → x == 0이면 ZF = 1
    je   .L6            # x == 0 → 바로 ret
    pushq %rbx          # callee-saved %rbx 저장
    movq %rdi, %rbx
    andl $1, %ebx       # %rbx = x & 1 (상위 32bit도 0)
    shrq %rdi           # %rdi = x >> 1 (재귀 인자)
    call pcount_r
    addq %rbx, %rax     # (x & 1) + 재귀 결과
    popq %rbx           # %rbx 되돌림
.L6:
    rep; ret
```

- 재귀 호출 때 %rdi가 x >> 1로 바뀌어 나가므로, 끝에서 더할 x & 1은 따로 보관 → callee-saved %rbx
- x == 0 경로는 %rbx를 건드리지 않으므로 pushq, popq를 건너뜀 (callee-saved는 수정할 때만)
- `rep; ret` : AMD 쪽 분기 최적화로 붙인 더미 1바이트 → 정확성과 무관, ret으로 읽어도 됨
- 호출마다 새 stack frame이 생겨 자기 return address와 자기 Saved %rbx를 따로 가짐 → 재귀 깊이만큼 stack이 자람
- 모든 호출이 같은 %rbx를 쓰지만 각 호출이 push/pop으로 되돌리므로 서로를 망가뜨리지 않음

손계산 7 · pcount_r(5): 5 = 101b → 기대 결과 2 (첫 진입 %rsp = 0x120, [0x120] = R0, 호출자 %rbx = B0, R1 = call 다음 줄 주소 — 주소는 16바이트 정렬 규약을 생략한 가상 값, 실제 ABI 진입 주소 아님)

호출 A (x = 5): %rdi = 5 ≠ 0 → pushq %rbx ([0x118] = B0), %rbx = 5 & 1 = 1, %rdi = 2, call → [0x110] = R1
호출 B (x = 2): 진입 %rsp = 0x110, pushq → [0x108] = 1 (A의 %rbx), %rbx = 2 & 1 = 0, %rdi = 1, call → [0x100] = R1
호출 C (x = 1): 진입 %rsp = 0x100, pushq → [0xf8] = 0 (B의 %rbx), %rbx = 1 & 1 = 1, %rdi = 0, call → [0xf0] = R1
호출 D (x = 0): 진입 직후 %rsp = 0xf0이다. %rax = 0이고, testq에서 ZF = 1이 되어 je .L6으로 이동한다. 이 경로에서는 %rbx를 저장하지 않으므로 pushq 없이 ret을 실행한다. ret 후 %rsp = 0xf8로 복귀한다. 최대 스택 사용량: 호출 A, B, C가 각각 saved %rbx 8바이트와 return address 8바이트를 사용하므로 총 3 × 16 = 48바이트이다. 즉, 최대 깊이에서는 %rsp = 0xf0이다.
되감기 C: addq %rbx, %rax → %rax = 0 + 1 = 1, popq → %rbx = 0, %rsp = 0x100, ret → 0x108
되감기 B: %rax = 1 + 0 = 1, popq → %rbx = 1, ret → %rsp = 0x118
되감기 A: %rax = 1 + 1 = 2, popq → %rbx = B0, %rsp = 0x120, ret → 0x128

결과

%rax = 2. 101b의 1 비트 개수와 일치. 각 호출이 보관한 %rbx: A = 1, B = 0, C = 1 (5의 비트를 LSB부터), 합 $1 + 0 + 1 = 2$

9 Stack과 호출 규약 정리

호출 규약이 보장하는 것 (슬라이드 71)

함수 호출에는 세 가지가 얽혀 있음: 제어 이동, 데이터 이동(인자·반환값), 저장 공간 관리(stack frame). 슬라이드 2의 세 메커니즘과 같은 축.

- **stack frame**: 호출마다 private storage (saved register, local 변수, saved return pointer)
- **register 저장 규약**: 한 함수 호출이 다른 함수의 데이터를 망가뜨리지 않게 함 (C 코드가 일부러 그러는 경우는 예외: Lecture 9의 buffer overflow)
- **stack discipline**: P가 Q를 호출하면 Q가 P보다 먼저 return → LIFO. mutual recursion(P ↔ Q)도 같은 규칙으로 처리

정리: caller-callee 관계와 context 저장이 축. 인자를 올바르게 다루고 caller가 나중에 쓸 register를 망가뜨리지 않는 것.

- 슬라이드 요약: 재귀(mutual 포함)도 보통의 호출 규약으로 처리 → local stack frame과 callee-saved register에 값을 안전하게 보관
- 슬라이드 요약 문구는 "인자는 stack top에 두고, 결과는 %rax로" — 정확히는 처음 6개 인자는 register(%rdi·%rsi·%rdx·%rcx·%r8·%r9)로 두고 7번째 이후만 stack top에 둠, 결과는 %rax로
- 포인터는 값의 주소(stack 또는 global)
- local 값은 stack frame에 두되 너무 크게 잡지 않음. stack은 계속 자라다 다른 영역과 겹칠 수 있음
- stack 크기에는 한도가 있음
- stack은 다른 데이터 영역과 반대 방향(낮은 주소)으로 자람

10 시험 포인트

- register를 읽는 것은 안전하고 쓰는 것이 문제: callee가 바꾼 register를 caller가 return 뒤에 다시 읽으면 값이 사라짐 → 보관 책임을 정해 둠
- caller-saved: caller가 call 전에 자기 frame에 저장하고 call 후 되돌림, callee는 마음대로 덮어씀
- callee-saved: callee가 쓰기 전에 저장하고 ret 전에 되돌림
- caller-saved: %rax, 인자 6개(%rdi %rsi %rdx %rcx %r8 %r9), %r10, %r11
- callee-saved: %rbx, %rbp, %r12~%r15, %rsp는 특수한 형태(나갈 때 원래 값으로)
- %rax는 반환값이라 callee가 반드시 덮어써서, 인자 register는 인자를 넘기려고 caller가 덮어써서 caller-saved
- call을 넘어 살아 있어야 하는 값은 callee-saved register(예: %rbx)에 두고, 그 register를 쓰는 함수는 push로 저장하고 pop으로 되돌림(수정할 때만)
- call_incr2 패턴: pushq %rbx → movq %rdi, %rbx → … → addq %rbx, %rax → popq %rbx, 되돌리기는 저장의 역순(LIFO)
- 저장·되돌림 장소는 stack만이 아님: 남는 register를 쓰는 편이 memory보다 빠름
- 재귀: 호출마다 stack frame이 생겨 자기 return address와 Saved %rbx를 가짐, mutual recursion까지 같은 규약
- pcount_r stack 사용: 종료가 아닌 호출 하나마다 16바이트 (return address 8 + Saved %rbx 8)
- rep; ret은 ret과 같은 동작, 정확성과 무관한 AMD용 최적화
- frame 구조: %rsp 쪽부터 Argument Build, Local variables + Saved registers, Old %rbp(optional), Return addr., 7번째 이후 인자(caller frame)
- stack은 낮은 주소 쪽으로 자라고 %rsp는 top 원소(가장 낮은 주소)의 주소
- pushq: %rsp를 8 줄이고 씀 / popq: 읽고 %rsp를 8 늘림, pop 뒤에도 값은 memory에 남고 %rsp가 유효한 top을 정함
- call label: return address(다음 instruction의 주소)를 push하고 점프
- ret: pop하고 그 주소로 점프 → call 직전 %rsp가 S이면 call 직후 S − 8, ret 직후 다시 S
- 인자 전달: 처음 6개는 %rdi → %rsi → %rdx → %rcx → %r8 → %r9, 7번째부터 stack, 반환값은 %rax

- 7번째 이후 인자는 caller의 frame에 있음: 진입 직후 8(%rsp), pushq를 할 때마다 offset이 8씩 커짐 (add10은 두 push 뒤 24(%rsp)가 a6)
- 재귀를 지원하려면 code가 re-entrant해야 하고, 호출마다 인자·local 변수·return pointer를 자기 frame에 둠
- frame은 진입할 때 할당하고 return할 때 해제, callee가 caller보다 먼저 return해서 LIFO로 충분
- call chain에서 frame은 호출 깊이만큼 쌓였다가 return하면 풀리고, 해제된 자리는 다음 호출이 다시 씀
- movl $imm, %esi는 %rsi 상위 32bit를 0으로 만듦 · 일반적인 인코딩에서 movq $imm32보다 2바이트 짧음 (5 vs 7바이트 — 인코딩 선택에 따라 다를 수 있음)
- call_incr 추적: subq $16 → 8(%rsp) = v1 = 15213 → leaq 8(%rsp), %rdi → incr 뒤 v1 = 18213, %rax = 15213 → addq 8(%rsp), %rax = 33426
- Small Exercise: a0~a5는 register, a6~a9는 stack, add10은 a5(%r9, caller-saved)를 첫 add5 호출 뒤에도 쓰려고 %rbp로 옮기고, callee-saved인 %rbx, %rbp는 push/pop으로 보존

11 연습문제

문제 1. 다음 register를 caller-saved와 callee-saved로 분류하라: %rax, %rdi, %rbx, %r10, %r12, %rbp, %r9

정답 보기

① caller-saved: %rax = 반환값, %rdi, %r9 (인자), %r10 (임시)

② callee-saved: %rbx, %r12, %rbp

최종 답 **caller-saved: %rax %rdi %r9 %r10 · callee-saved: %rbx %r12 %rbp**

문제 2. who가 %rax에 100을 반환한다고 하자 (본문 yoo/who). yoo가 돌려주는 값은? 그리고 yoo가 15313을 돌려주도록 고치는 방법을 두 가지 말하라.

정답 보기

① %rdx = $15213 - 18213 = -3000$ → %rax = $100 + (-3000) = -2900$

② 방법 1 (caller-saved 방식): yoo가 call 전에 pushq %rdx, call 뒤에 popq %rdx로 저장·되돌림 → addq %rdx, %rax가 15213을 더함

③ 방법 2 (callee-saved register): 15213을 %rbx에 두고 (yoo 자신의 caller를 위해 pushq %rbx … popq %rbx) call 뒤에 addq %rbx, %rax → who가 %rbx를 쓰더라도 되돌려 줌

최종 답 **그대로면 −2900 · 두 방법 모두** $100 + 15213 = 15313$

문제 3. call_incr2(7)에서 진입 직전 %rsp = 0x200이라고 하자 (주소는 정렬 규약을 생략한 가상 값, 실제 ABI 진입 주소 아님). ① call incr 직후의 %rsp ② incr 진입 시 %rdi ③ incr가 끝난 뒤 v1의 값 ④ 반환값을 구하라.

정답 보기

① pushq → %rsp = 0x1f8, subq $16 → %rsp = 0x1e8, v1 = 8(%rsp) → 주소 0x1f0

② ① call이 return address를 push → %rsp = 0x1e0

③ ② %rdi = &v1 = 0x1f0

④ ③ v1 = 15213 + 3000 = 18213

⑤ ④ 반환값 = 7 + 15213 = 15220

최종 답 **① 0x1e0 ② 0x1f0 ③ 18213 ④ 15220**

문제 4. call_incr2에서 movq %rdi, %rbx를 없애고 addq %rbx, %rax 대신 addq %rdi, %rax를 쓰면 왜 틀리는가?

문제 3과 같은 조건(x = 7, 진입 직전 %rsp = 0x200)에서 반환값은?

정답 보기

① call 직전 leaq 8(%rsp), %rdi로 %rdi가 x가 아니라 &v1 = 0x1f0이 됨 (인자 register는 caller-saved — 규약만으로는 보존이 보장되지 않지만, 제시된 incr 코드는 %rdi를 그대로 인자로 사용한다고 가정)

② %rax = $15213 + \texttt{0x1f0} = 15213 + 496 = 15709$

최종 답 **15709. 올바른 값 15220과 다름 (x 대신 주소를 더함)**

문제 5. pcount_r에서 ① 호출 횟수(최초 호출 포함) ② 호출마다 %rbx에 보관되는 x & 1 ③ 첫 진입 시점보다 가장 깊을 때 stack이 더 쓰는 바이트 수 ④ 반환값을 구하라.

정답 보기

① 6 = 110b → x = 6, 3, 1, 0 → ① 4번

② ② x = 6: 0, x = 3: 1, x = 1: 1 (x = 0 호출은 %rbx를 건드리지 않음)

③ ③ 종료가 아닌 호출 3번 × 16바이트 (saved %rbx 8 + return address 8) = 48바이트

④ ④ 되감기: 0 → 0 + 1 = 1 → 1 + 1 = 2 → 2 + 0 = 2

최종 답 **① 4번 ② 0, 1, 1 ③ 48바이트 ④ 2**

문제 6. pcount_r의 호출 횟수와 가장 깊을 때 추가 stack 사용량은? $x = 2^{64} - 1$이면 어떻게 되는가?

정답 보기

① 255 = 11111111b → x = 255, 127, 63, 31, 15, 7, 3, 1, 0 → 9번 호출, 반환값 8

② 종료가 아닌 호출 8번 × 16바이트 = 128바이트

③ $x = 2^{64} - 1$: 64비트가 모두 1 → x가 64번 줄어든 뒤 0이 되므로 65번 호출, $64 \times 16 = 1024$ 바이트

최종 답 **9번 · 128바이트 (x = 2⁶⁴−1이면 65번 · 1024바이트)**

문제 7. pcount_r에서 x == 0이면 pushq %rbx를 건너뛰고 바로 .L6로 가도 되는 이유는? 이때 반환값 0은 어디에서 정해지는가?

정답 보기

① callee-saved register는 callee가 값을 수정할 때만 저장했다가 되돌리면 됨 → 이 경로는 %rbx를 건드리지 않으므로 push/pop이 필요 없음

② 반환값 0은 함수 맨 앞의 movl $0, %eax에서 이미 정해짐 (32bit 목적지라 %rax 전체가 0), je .L6 뒤 rep; ret이 그대로 돌려줌

최종 답 **%rbx를 수정하지 않는 경로라 저장 불필요 · 0은 movl $0, %eax에서 설정**

문제 8. (슬라이드 73~74 Small Exercise)

```
long add5(long b0, long b1, long b2, long b3, long b4) {
    return b0 + b1 + b2 + b3 + b4;
}
long add10(long a0, long a1, long a2, long a3, long a4,
           long a5, long a6, long a7, long a8, long a9) {
    return add5(a0, a1, a2, a3, a4) + add5(a5, a6, a7, a8, a9);
}
```

① a0~a9는 어디로 전달되는가 ② b0~b4는 어디로 전달되는가 ③ add10에서 저장해야 하는 register는?

정답 보기

① ① a0~a5는 %rdi, %rsi, %rdx, %rcx, %r8, %r9 · a6~a9는 stack

② ② b0~b4는 %rdi, %rsi, %rdx, %rcx, %r8 (인자가 5개라 register만으로 충분)

③ ③ 질문만으로는 정해지지 않고 assembly를 봐야 함 (슬라이드: ill-posed question) → 실제로는 %rbx, %rbp (callee-saved라 push/pop으로 보존) + %r9의 값 a5 (caller-saved라 %rbp로 이동해 보존) (첫 번째 add5를 부르는 동안)

④ %r9: a5가 들어 있음. 인자 register는 caller-saved라 add5가 덮어써도 되므로 두 번째 add5에 줄 a5를 movq %r9, %rbp로 미리 옮겨 둠

⑤ %rbp, %rbx: callee-saved인데 add10이 쓰므로 진입할 때 pushq로 저장하고 ret 전에 popq로 되돌림 (%rbp에는 a5, %rbx에는 첫 add5의 반환값 보관)

⑥ a6~a9는 stack에 있어서 register에 따로 저장할 필요 없음

최종 답 **③ %rbx, %rbp (+ a5를 옮겨 둘 곳으로 %rbp 사용)**

문제 9. 문제 8의 add10을 컴파일한 assembly (슬라이드 74)

```
add10:
    pushq %rbp
    pushq %rbx
    movq  %r9, %rbp
    call  add5
    movq  %rax, %rbx
    movq  48(%rsp), %r8
    movq  40(%rsp), %rcx
    movq  32(%rsp), %rdx
    movq  24(%rsp), %rsi
    movq  %rbp, %rdi
    call  add5
    addq  %rbx, %rax
    popq  %rbx
    popq  %rbp
    ret
```

add10에 들어온 직후 %rsp = 0x200이라고 하자 ([0x200]에 return address; 주소는 정렬 규약을 생략한 가상 값, 실제 ABI 진입 주소 아님). ① 두 pushq 뒤 %rsp ② a6, a7, a8, a9가 있는 주소 ③ 두 번째 call add5 직전 %rdi, %rsi, %rdx, %rcx, %r8에 들어 있는 것 ④ 두 번째 call add5가 return address를 push한 직후 %rsp

정답 보기

① ① %rsp = $0x200 - 16 = \texttt{0x1f0}$ ([0x1f8] = 옛 %rbp, [0x1f0] = 옛 %rbx)

② ② 들어온 직후 a6 = 8(%rsp) → 0x208, a7 = 0x210, a8 = 0x218, a9 = 0x220 (7번째 이후 인자는 return address 위쪽). 두 push 뒤에는 %rsp 기준 24, 32, 40, 48: $0x1f0 + 24 = \texttt{0x208}$ 이라서 24(%rsp)가 a6, 48(%rsp)가 a9

③ ③ %rdi = a5 (%rbp에서), %rsi = a6, %rdx = a7, %rcx = a8, %r8 = a9

④ ④ %rsp = $0x1f0 - 8 = \texttt{0x1e8}$

최종 답 **① 0x1f0 ② a6 = 0x208 … a9 = 0x220 ③ a5, a6, a7, a8, a9 ④ 0x1e8**

문제 10. 문제 9의 add10을 add10(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)으로 부른다.

- ① 첫 번째 add5의 반환값과 그것을 보관하는 register
- ② %rbp의 값
- ③ 두 번째 add5에서 addq %rsi, %rdi / addq %rdi, %rdx / addq %rdx, %rcx / leaq (%rcx,%r8), %rax를 차례로 실행한 뒤의 %rdi, %rdx, %rcx, %rax
- ④ add10의 반환값

```
add5:
    addq %rsi, %rdi
    addq %rdi, %rdx
    addq %rdx, %rcx
    leaq (%rcx,%r8), %rax
    ret
```

정답 보기

① ① add5(1, 2, 3, 4, 5): %rdi = 1 + 2 = 3, %rdx = 3 + 3 = 6, %rcx = 4 + 6 = 10, %rax = 10 + 5 = 15 → 반환값 15, %rbx에 보관 (movq %rax, %rbx)

② ② %rbp = a5 = 6 (movq %r9, %rbp)

③ ③ add5(6, 7, 8, 9, 10): %rdi = 6 + 7 = 13, %rdx = 8 + 13 = 21, %rcx = 9 + 21 = 30, %rax = 30 + 10 = 40

④ ④ addq %rbx, %rax → $15 + 40 = 55$

최종 답 **① 15, %rbx ② 6 ③ 13, 21, 30, 40 ④ 55 (1부터 10까지의 합과 같음)**

문제 11. callq 400700 <f>가 주소 0x400620에 있고 5바이트 길이이다. call 직전 %rsp = 0x200이고 f는 pushq %rbx로 시작해 popq %rbx 뒤 retq로 끝난다.

- ① call 직후의 %rsp, %rip, stack top의 값
- ② f의 pushq %rbx 직후 %rsp
- ③ retq 직전 %rsp
- ④ retq 직후 %rsp, %rip

정답 보기

① ① %rsp = $0x200 - 8 = \texttt{0x1f8}$, %rip = 0x400700, [0x1f8] = $0x400620 + 5 = \texttt{0x400625}$ (call 다음 instruction의 주소)

② ② 0x1f0

③ ③ popq %rbx로 8 늘어난 0x1f8 (stack top에 return address가 있어야 함)

④ ④ %rsp = 0x200, %rip = 0x400625

최종 답 **① 0x1f8 / 0x400700 / 0x400625 ② 0x1f0 ③ 0x1f8 ④ 0x200 / 0x400625**

문제 12. 본문 call chain에서 yoo는 who를 부르고, who는 amI를 두 번 부르며, 첫 번째 amI는 amI를 부르고 그 amI가 다시 amI를 부른다.

- ① 호출(call) 횟수
- ② 가장 깊을 때 stack의 frame 수(yoo 포함)
- ③ 호출 하나가 stack을 16바이트 쓰고 yoo 실행 중 %rsp = 0x120이면 가장 깊을 때 %rsp
- ④ who가 두 번째 amI를 부를 때 그 frame의 %rsp

정답 보기

① ① who 1번 + amI 4번 (who가 부른 2번, 재귀로 부른 2번) = 5번

② ② yoo, who, amI 3개 = 5개

③ ③ $0x120 - 4 \times 16 = 0x120 - 64 =$ `0xe0` (who 1개 + amI 3개가 쌓인 상태)

④ ④ 0x100: who frame의 %rsp가 0x110이므로 16바이트 아래. 첫 번째 amI가 쓰던 자리를 다시 씀

최종 답 **① 5번 ② 5개 ③ 0xe0 ④ 0x100**